

CS-29: Advance Java Programming (J2EE) - Practical Solutions

All 78 Programming Exercises - Solved with Explanation, Program & Output

Created by Nenshi

Each solution below follows the same format: the exercise concept/definition, a short (as concise as possible) Java/JSP/Servlet program, its expected output or browser/console result, and a short explanation of the logic or flow. Programs needing a running server, browser, or live database show representative output since actual results depend on your server setup and data.

Index of Exercises

Q. No.	Exercise
1	Write a program to insert a record of an employee into the emp table.
2	Write a program to display all the records of employees.
3	Write a program to display employees whose salary is greater than 50000.
4	Write a JDBC program to display employees who are from the city 'Rajkot'.
5	Write a program to display employees whose name starts with 'A'.
6	Write a program to display employees whose designation is manager.
7	Write a program to count the number of employees in the table.
8	Write a program to display the employee with the highest salary.
9	Write a program to sort employee records by empnm.
10	Write a program which accepts empno from the user and displays the corresponding employee record.
11	Write a program which accepts a department name and displays employee name with designation.
12	Write a Program which inserts a record of a student by using Prepared Statement.
13	Write a Program which updates a record of a student by using Prepared Statement.
14	Write a Program to delete a record of a student whose rollno is given by the user.
15	Write a program which inserts a default record of an employee using a callable statement (Procedure without parameter).
16	Write a program which inserts a record of an employee using a callable statement (Procedure with Parameter).
17	Write a program which displays employee's designation by providing empno using callable statement.
18	Write a program which displays all records of employees whose designation is provided by the user using a callable statement.
19	Write a program that performs CRUD operations on the product table (pid, productname, price, quantity).
20	Write a program to connect to a database and retrieve metadata.
21	Write a servlet to print Hello World.
22	Write a servlet that displays today's date and time.
23	Write a program that accepts username and after submitting the form, welcomes the user by writing 'Welcome Username'.
24	Write a program that accepts Employee Number, Name, Designation, Qualifications and displays it on a page using GET method.
25	Write a program that registers a student (rollno, name, course, semester, hobbies) using POST and displays info using getParameterNames() and getParameterValues().
26	Demonstrate the Servlet lifecycle with appropriate messages in each stage.
27	Write a servlet program that displays the Basic Header Information.

Q. No.	Exercise
28	Write a program that creates a login form; if login details (admin/admin) are correct, display welcome, else display an error message.
29	Write a Servlet Program that gives Addition, Subtraction, Multiplication, Division options using radio buttons.
30	Write a program that redirects a page to google.com.
31	Write a program that sends a customized error message for '404 - Page Not Found'.
32	Write a servlet program that demonstrates the use of URL Rewriting.
33	Write a servlet program that demonstrates the use of Hidden Form Field.
34	Write a servlet program that checks if a cookie exists; if not, creates one, and displays all cookies sent by the browser.
35	Write a servlet that creates a 'username' cookie (1 day max age), adds it to the response, and displays all cookies received.
36	Write a servlet that displays 'Welcome' on first visit and 'Welcome back' on later visits, using Cookie.
37	Develop a servlet that lets the user pick a background color from a dropdown, saves it in a cookie, and applies it to the response page.
38	Write a servlet that displays 'Welcome'/'Welcome back' using HttpSession instead of Cookie.
39	Write a servlet that accepts a user's name via a form and stores it in an HTTP session; on later visits during the session, greet the user by name.
40	Write a Java Servlet application that authenticates users (login form), stores username in session, shows a personalized welcome page, and implements logout (invalidates session).
41	Create a servlet that counts how many times a user has visited the page during a session; display the visit count on each load.
42	Develop a servlet that lets the user pick a background color from a dropdown, saves it in the session, and applies it to the response page.
43	Write a servlet program that checks if a user has logged in; if not, redirect to a login form. Once logged in, store username in session and display a welcome message.
44	Write a JSP to print Hello World.
45	Write a JSP that accepts Student's details from the HTML page and displays all the information.
46	Write a JSP that accepts a year from the user and prints whether it is a leap year or not.
47	Write a JSP program to calculate simple interest by accepting values from the user.
48	Write a JSP page that performs division; if divisor is zero, handle the exception using a custom error page.
49	Configure an error page in web.xml that catches HTTP 404 errors and displays a user-friendly message.
50	Write a JSP application that uses <jsp:include> to include reusable pages such as header and footer into a main content page.
51	Create a JSP application where the user enters a favorite color; use <jsp:forward> in process.jsp to forward to blue.jsp if input is 'blue', otherwise to default.jsp.
52	Create a JSP page that uses <jsp:useBean> to instantiate a User JavaBean (name, email, age) and display the user details.
53	Create a JavaBean EmployeeBean (name, designation, salary); instantiate with <jsp:useBean> scope session, set properties with <jsp:setProperty>, and display using EL (\${}).
54	Write a JSP program to demonstrate the use of application implicit objects - maintain a global visitor counter that increments on every access.
55	Write a JSP program with login (login.jsp), session-based welcome (welcome.jsp), and logout (logout.jsp) that invalidates the session.
56	Write a JSP program that stores the user's preferred theme (light/dark) using cookies; retrieve it on subsequent visits and apply it.
57	Write a JSP that displays 'Welcome' on first visit and 'Welcome back' on later visits, using cookies.
58	Write a JSP program to display current date and time using JSP Expression Language (EL).

Q. No.	Exercise
59	Write a JSP application using EL for dynamic content across pages: index.jsp takes a name, passes it via request to welcome.jsp, which displays it using EL.
60	Write a JSP program that calculates addition, subtraction, multiplication and division using Expression Language and displays results dynamically.
61	Write a JSP program to check whether it is a leap year or not using JSTL.
62	Write a JSP program to check whether it is an even or odd number using JSTL.
63	Write a JSP program to print prime numbers from 1 to 100 using JSTL.
64	Write a JSP program to display whether a number is divisible by 2 using JSTL.
65	Write a program with a Servlet that accepts Username as input; use a JSP page to welcome the user. Use the MVC pattern to separate concerns.
66	Write a login program (admin/admin) using MVC with 2 separate views - one for success, one for failure.
67	Write a login program (Table: Login, fields username/password) using MVC, fetching credentials from the database.
68	Develop a Registration module using JSP, Servlet and MVC with database connectivity (Table: User - username, password, mobile).
69	Write a program that lets a user request a list of students by course and semester; use a Servlet to retrieve data from a Model and display it dynamically on a JSP page. (Table: Student)
70	Write a Contact Management System using MVC where users can add contacts (name, phone) and view all saved contacts.
71	Write an MVC application using Servlet, JSP and JavaBean to insert and display employee data (emp table: empno, empnm, designation, dept).
72	Develop a full CRUD application using MVC (Servlet, JSP, JavaBean) for student records (studnt table: rollno, name, course, semester).
73	Create a simple Hibernate application connecting to MySQL: define a Student entity (rollno, name, course, semester), save a new object and retrieve it using a session.
74	Create an Employee entity (empno, empnm, department); use Hibernate to insert a record, then retrieve and display all employees.
75	Create a Hibernate application with a Product entity (id, name, price); use HQL to fetch all products and display id, name, price.
76	Create a simple login application using the Spring framework with a LoginController managing authentication via Spring MVC, with login form, success and failure templates.
77	Develop an application using Spring Boot and Spring Data JPA to insert and display employee data (Employee table: empno, empnm, designation, dept).
78	Develop an application using Spring Boot and Spring Data JPA to perform CRUD operations on a Student table (rollno, name, course, semester).

Setup: Common JDBC Connection Helper (used by all JDBC programs Q1-Q20)

Definition / Concept:

A single reusable class opens the database connection so every JDBC program below can simply call `DBUtil.getConnection()` instead of repeating the same driver/URL code each time.

Program:

```
// DBUtil.java
import java.sql.*;
public class DBUtil {
    public static Connection getConnection() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb", "root", "password");
    }
}
// Table emp : empno, empnm, designation, city, salary, department
// Table stud : rollno, firstname, lastname, course, semester
```

Output:

(No direct output - this class is reused by every JDBC program below.)

Q1. Write a program to insert a record of an employee into the emp table.

Definition / Concept:

A Statement (or PreparedStatement) executes an SQL INSERT command to add one new row into the table.

Program:

```
import java.sql.*;
public class InsertEmp {
    public static void main(String[] a) throws Exception {
        Connection con = DBUtil.getConnection();
        Statement st = con.createStatement();
        st.executeUpdate("INSERT INTO emp VALUES(101, 'Ravi', 'Manager', 'Rajkot', 55000, 'IT')");
        System.out.println("Record inserted successfully.");
        con.close();
    }
}
```

Output:

Record inserted successfully.

Q2. Write a program to display all the records of employees.

Definition / Concept:

`SELECT *` retrieves every row and column. The `ResultSet` is iterated row by row using `next()`, and each column is read with `getXxx(columnIndex)`.

Program:

```
import java.sql.*;
public class DisplayEmp {
    public static void main(String[] a) throws Exception {
        Connection con = DBUtil.getConnection();
        ResultSet rs = con.createStatement().executeQuery("SELECT * FROM emp");
        while (rs.next())
            System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3)+
                               " "+rs.getString(4)+" "+rs.getDouble(5)+" "+rs.getString(6));
        con.close();
    }
}
```

Output:

101 Ravi Manager Rajkot 55000.0 IT
102 Asha Clerk Surat 30000.0 Admin

Q3. Write a program to display employees whose salary is greater than 50000.

Definition / Concept:

A WHERE clause filters rows so only matching records are returned by the query.

Program:

```
ResultSet rs = con.createStatement().executeQuery(  
    "SELECT * FROM emp WHERE salary > 50000");  
while (rs.next())  
    System.out.println(rs.getString("empnm") + " - " + rs.getDouble("salary"));
```

Output:

Ravi - 55000.0

Q4. Write a JDBC program to display employees who are from the city 'Rajkot'.

Definition / Concept:

The WHERE clause compares the city column against a fixed string value.

Program:

```
ResultSet rs = con.createStatement().executeQuery(  
    "SELECT * FROM emp WHERE city='Rajkot'");  
while (rs.next())  
    System.out.println(rs.getString("empnm") + " - " + rs.getString("city"));
```

Output:

Ravi - Rajkot

Q5. Write a program to display employees whose name starts with 'A'.

Definition / Concept:

The SQL LIKE operator with pattern 'A%' matches any name beginning with the letter A.

Program:

```
ResultSet rs = con.createStatement().executeQuery(  
    "SELECT * FROM emp WHERE empnm LIKE 'A%'");  
while (rs.next())  
    System.out.println(rs.getString("empnm"));
```

Output:

Asha

Q6. Write a program to display employees whose designation is manager.

Definition / Concept:

An equality condition on the designation column filters only rows where designation = 'Manager'.

Program:

```
ResultSet rs = con.createStatement().executeQuery(  
    "SELECT * FROM emp WHERE designation='Manager'");  
while (rs.next())  
    System.out.println(rs.getString("empnm"));
```

Output:

Ravi

Q7. Write a program to count the number of employees in the table.

Definition / Concept:

The SQL aggregate function COUNT(*) returns the total number of rows in the table as a single value.

Program:

```
ResultSet rs = con.createStatement().executeQuery("SELECT COUNT(*) FROM emp");
if (rs.next())
    System.out.println("Total employees: " + rs.getInt(1));
```

Output:

Total employees: 5

Q8. Write a program to display the employee with the highest salary.

Definition / Concept:

Sorting all rows by salary in descending order (ORDER BY salary DESC) and taking only the first row (LIMIT 1) gives the highest-paid employee.

Program:

```
ResultSet rs = con.createStatement().executeQuery(
    "SELECT * FROM emp ORDER BY salary DESC LIMIT 1");
if (rs.next())
    System.out.println(rs.getString("empnm") + " - " + rs.getDouble("salary"));
```

Output:

Ravi - 55000.0

Q9. Write a program to sort employee records by empnm.

Definition / Concept:

ORDER BY empnm arranges the returned rows alphabetically by the employee name column.

Program:

```
ResultSet rs = con.createStatement().executeQuery(
    "SELECT * FROM emp ORDER BY empnm");
while (rs.next())
    System.out.println(rs.getString("empnm"));
```

Output:

Asha
Ravi

Q10. Write a program which accepts empno from the user and displays the corresponding employee record.

Definition / Concept:

A PreparedStatement with a ? placeholder safely inserts the user-entered value into the query. Scanner reads the empno typed at the console.

Program:

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
System.out.print("Enter empno: ");
int eno = sc.nextInt();
PreparedStatement ps = con.prepareStatement("SELECT * FROM emp WHERE empno=?");
ps.setInt(1, eno);
ResultSet rs = ps.executeQuery();
```

```
if (rs.next())
    System.out.println(rs.getString("empnm") + " - " + rs.getString("designation"));
```

Output:

```
Enter empno: 101
Ravi - Manager
```

Q11. Write a program which accepts a department name and displays employee name with designation.

Definition / Concept:

Only the two needed columns (empnm, designation) are selected, filtered by the department value supplied by the user.

Program:

```
PreparedStatement ps = con.prepareStatement(
    "SELECT empnm, designation FROM emp WHERE department=?");
ps.setString(1, "IT");
ResultSet rs = ps.executeQuery();
while (rs.next())
    System.out.println(rs.getString(1) + " - " + rs.getString(2));
```

Output:

```
Ravi - Manager
```

Q12. Write a Program which inserts a record of a student by using Prepared Statement.

Definition / Concept:

PreparedStatement pre-compiles the SQL once; ? placeholders are filled in with setXxx() methods, which is safer and faster than building SQL strings manually.

Program:

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO stud VALUES(?,?,?,?,?)");
ps.setInt(1, 1);
ps.setString(2, "Raj");
ps.setString(3, "Patel");
ps.setString(4, "BCA");
ps.setInt(5, 5);
ps.executeUpdate();
System.out.println("Student record inserted.");
```

Output:

```
Student record inserted.
```

Q13. Write a Program which updates a record of a student by using Prepared Statement.

Definition / Concept:

An UPDATE statement with ? placeholders changes specific column(s) of the row matching the given rollno.

Program:

```
PreparedStatement ps = con.prepareStatement(
    "UPDATE stud SET course=? WHERE rollno=?");
ps.setString(1, "BCA-Hons");
ps.setInt(2, 1);
ps.executeUpdate();
System.out.println("Student record updated.");
```

Output:

Student record updated.

Q14. Write a Program to delete a record of a student whose rollno is given by the user.

Definition / Concept:

The rollno entered by the user is bound to a ? placeholder in a DELETE statement, removing only that specific row.

Program:

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter rollno to delete: ");
int r = sc.nextInt();
PreparedStatement ps = con.prepareStatement("DELETE FROM stud WHERE rollno=?");
ps.setInt(1, r);
ps.executeUpdate();
System.out.println("Student record deleted.");
```

Output:

```
Enter rollno to delete: 1
Student record deleted.
```

Q15. Write a program which inserts a default record of an employee using a callable statement (Procedure without parameter).

Definition / Concept:

A CallableStatement invokes a stored procedure using {call procedureName()}. Since the procedure takes no parameters, it always inserts the same fixed (default) record.

Program:

```
-- SQL: CREATE PROCEDURE insertDefaultEmp()
--      BEGIN INSERT INTO emp VALUES(999,'Default','Staff','Rajkot',20000,'General'); END;

CallableStatement cs = con.prepareCall("{call insertDefaultEmp()}");
cs.execute();
System.out.println("Default employee inserted.");
```

Output:

```
Default employee inserted.
```

Q16. Write a program which inserts a record of an employee using a callable statement (Procedure with Parameter).

Definition / Concept:

IN parameters let the caller pass values into the stored procedure using ? placeholders, similar to a PreparedStatement.

Program:

```
-- SQL: CREATE PROCEDURE insertEmp(IN eno INT, IN enm VARCHAR(50), IN desig VARCHAR(30),
--      IN cty VARCHAR(30), IN sal DOUBLE, IN dept VARCHAR(30))
--      BEGIN INSERT INTO emp VALUES(enno,enm,desig,cty,sal,dept); END;

CallableStatement cs = con.prepareCall("{call insertEmp(?,?,?,?,?,?)}");
cs.setInt(1, 103); cs.setString(2, "Kiran"); cs.setString(3, "Clerk");
cs.setString(4, "Jamnagar"); cs.setDouble(5, 28000); cs.setString(6, "Admin");
cs.execute();
System.out.println("Employee inserted via procedure.");
```

Output:

Employee inserted via procedure.

Q17. Write a program which displays employee's designation by providing empno using callable statement.

Definition / Concept:

An OUT parameter lets the stored procedure send a value back to the Java program. registerOutParameter() declares its SQL type before execution, and getXxx() reads it afterwards.

Program:

```
-- SQL: CREATE PROCEDURE getDesignation(IN eno INT, OUT desig VARCHAR(30))
--      BEGIN SELECT designation INTO desig FROM emp WHERE empno=eno; END;

CallableStatement cs = con.prepareCall("{call getDesignation(?,?)}");
cs.setInt(1, 101);
cs.registerOutParameter(2, java.sql.Types.VARCHAR);
cs.execute();
System.out.println("Designation: " + cs.getString(2));
```

Output:

Designation: Manager

Q18. Write a program which displays all records of employees whose designation is provided by the user using a callable statement.

Definition / Concept:

MySQL procedures can return a result set directly from a SELECT statement inside them; executeQuery() on the CallableStatement retrieves that result set.

Program:

```
-- SQL: CREATE PROCEDURE getEmpByDesignation(IN desig VARCHAR(30))
--      BEGIN SELECT * FROM emp WHERE designation=desig; END;

CallableStatement cs = con.prepareCall("{call getEmpByDesignation(?)}");
cs.setString(1, "Manager");
ResultSet rs = cs.executeQuery();
while (rs.next())
    System.out.println(rs.getString("empnm"));
```

Output:

Ravi

Q19. Write a program that performs CRUD operations on the product table (pid, productname, price, quantity).

Definition / Concept:

CRUD stands for Create, Read, Update, Delete - the four basic database operations, each implemented here as a short PreparedStatement method.

Program:

```
class ProductDAO {
    void insert(Connection con) throws Exception {
        PreparedStatement ps = con.prepareStatement("INSERT INTO product VALUES(?,?,?,?)");
        ps.setInt(1,1); ps.setString(2,"Pen"); ps.setDouble(3,10.0); ps.setInt(4,100);
        ps.executeUpdate();
    }
    void read(Connection con) throws Exception {
        ResultSet rs = con.createStatement().executeQuery("SELECT * FROM product");
```

```

        while (rs.next()) System.out.println(rs.getString("productname"));
    }
    void update(Connection con) throws Exception {
        PreparedStatement ps = con.prepareStatement("UPDATE product SET price=? WHERE pid=?");
        ps.setDouble(1,12.0); ps.setInt(2,1); ps.executeUpdate();
    }
    void delete(Connection con) throws Exception {
        PreparedStatement ps = con.prepareStatement("DELETE FROM product WHERE pid=?");
        ps.setInt(1,1); ps.executeUpdate();
    }
}

```

Output:

```

Pen
(insert/update/delete each print the affected-row count internally;
read() prints the current product list)

```

Q20. Write a program to connect to a database and retrieve metadata.

Definition / Concept:

DatabaseMetaData describes the database itself (product name, version, driver) rather than table data, useful for diagnostics.

Program:

```

DatabaseMetaData md = con.getMetaData();
System.out.println("Product : " + md.getDatabaseProductName());
System.out.println("Version : " + md.getDatabaseProductVersion());
System.out.println("Driver : " + md.getDriverName());

```

Output:

```

Product : MySQL
Version : 8.0.36
Driver : MySQL Connector/J

```

Q21. Write a servlet to print Hello World.

Definition / Concept:

A Servlet is a Java class that extends HttpServlet and overrides doGet()/doPost() to handle HTTP requests. It writes HTML directly to the response using PrintWriter, which the browser then renders.

Program:

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h1>Hello World</h1>");
    }
}
// web.xml maps a URL pattern (e.g. /hello) to this servlet class.

```

Output:

```

Browser displays:
Hello World (rendered as an <h1> heading)

```

Q22. Write a servlet that displays today's date and time.

Definition / Concept:

java.util.Date's toString() gives the current date and time, which is written straight into the HTML response.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    res.setContentType("text/html");
    PrintWriter out = res.getWriter();
    out.println("<h2>Today: " + new java.util.Date() + "</h2>");
}
```

Output:

Browser displays:
Today: Sun Sep 13 16:45:02 IST 2026

Q23. Write a program that accepts username and after submitting the form, welcomes the user by writing 'Welcome Username'.

Definition / Concept:

An HTML form with method='get' sends the typed value as a URL query parameter, which the servlet reads with request.getParameter().

Program:

```
<!-- form.html -->
<form action="welcome" method="get">
    Username: <input type="text" name="username">
    <input type="submit" value="Submit">
</form>

// WelcomeServlet.java
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    String uname = req.getParameter("username");
    res.getWriter().println("Welcome " + uname);
}
```

Output:

Browser displays:
Welcome Ravi

Q24. Write a program that accepts Employee Number, Name, Designation, Qualifications and displays it on a page using GET method.

Definition / Concept:

Multiple form fields are each read individually with request.getParameter(fieldName) and printed together on the response page.

Program:

```
<!-- empform.html -->
<form action="empinfo" method="get">
    EmpNo: <input name="empno">
    Name: <input name="empnm">
    Designation: <input name="designation">
    Qualification: <input name="qualification">
    <input type="submit">
</form>

// EmpInfoServlet.java
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    PrintWriter out = res.getWriter();
    out.println(req.getParameter("empno") + " " + req.getParameter("empnm") + " " +
        req.getParameter("designation") + " " + req.getParameter("qualification"));
}
```

Output:

Browser displays:
101 Ravi Manager MCA

Q25. Write a program that registers a student (rollno, name, course, semester, hobbies) using POST and displays info using `getParameterNames()` and `getParameterValues()`.

Definition / Concept:

`getParameterNames()` returns every field name submitted by the form (useful when field names/count aren't fixed, e.g. multiple hobby checkboxes), and `getParameterValues()` returns all values for a given name since a checkbox field can have multiple selections.

Program:

```
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
    PrintWriter out = res.getWriter();
    java.util.Enumeration<String> names = req.getParameterNames();
    while (names.hasMoreElements()) {
        String pname = names.nextElement();
        String[] vals = req.getParameterValues(pname);
        out.println(pname + ": " + String.join(", ", vals));
    }
}
```

Output:

rollno: 1
name: Raj
course: BCA
semester: 5
hobbies: Reading,Music

Q26. Demonstrate the Servlet lifecycle with appropriate messages in each stage.

Definition / Concept:

A servlet's lifecycle has 3 stages: `init()` runs once when the servlet is first loaded, `service()/doGet()` runs for every request, and `destroy()` runs once when the servlet is being removed/server shuts down.

Program:

```
public class LifecycleServlet extends HttpServlet {
    public void init() { System.out.println("init() - Servlet initialized"); }

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        System.out.println("service() - Handling a request");
        res.getWriter().println("Servlet is running");
    }

    public void destroy() { System.out.println("destroy() - Servlet destroyed"); }
}
```

Output:

Server console:
init() - Servlet initialized
service() - Handling a request (printed on every request)
destroy() - Servlet destroyed (printed when the server stops)

Q27. Write a servlet program that displays the Basic Header Information.

Definition / Concept:

getHeaderNames() lists every HTTP header sent by the browser (like User-Agent, Host, Accept-Language);
getHeader(name) fetches each one's value.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    PrintWriter out = res.getWriter();
    java.util.Enumeration<String> heads = req.getHeaderNames();
    while (heads.hasMoreElements()) {
        String h = heads.nextElement();
        out.println(h + ": " + req.getHeader(h) + "<br>");
    }
}
```

Output:

Browser displays:
host: localhost:8080
user-agent: Mozilla/5.0 ...
accept-language: en-US,en;q=0.9

Q28. Write a program that creates a login form; if login details (admin/admin) are correct, display welcome, else display an error message.

Definition / Concept:

The submitted username/password are compared against fixed values with equals(). A conditional branch then writes either the welcome or the error HTML.

Program:

```
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
    String u = req.getParameter("username");
    String p = req.getParameter("password");
    PrintWriter out = res.getWriter();
    if (u.equals("admin") && p.equals("admin"))
        out.println("<h2>Welcome, " + u + "!</h2>");
    else
        out.println("<h2>Invalid username or password</h2>");
}
```

Output:

Browser displays (correct credentials):
Welcome, admin!

Browser displays (wrong credentials):
Invalid username or password

Q29. Write a Servlet Program that gives Addition, Subtraction, Multiplication, Division options using radio buttons.

Definition / Concept:

The selected radio button's value is read as a normal request parameter; a switch statement then performs the matching arithmetic operation.

Program:

```
<!-- calc.html -->
<form action="calc" method="post">
    A: <input name="a"> B: <input name="b">
    <input type="radio" name="op" value="+">Add
    <input type="radio" name="op" value="-">Sub
    <input type="radio" name="op" value="*">Mul
    <input type="radio" name="op" value="/">Div
    <input type="submit">
</form>
```

```
// CalcServlet.java
double a = Double.parseDouble(req.getParameter("a"));
double b = Double.parseDouble(req.getParameter("b"));
String op = req.getParameter("op");
double result = switch (op) {
    case "+" -> a+b; case "-" -> a-b; case "*" -> a*b; default -> a/b;
};
res.getWriter().println("Result: " + result);
```

Output:

Browser displays:
Result: 13.0

Q30. Write a program that redirects a page to google.com.

Definition / Concept:

sendRedirect() sends an HTTP 302 response telling the browser to load a different URL - the browser itself then navigates there.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    res.sendRedirect("https://www.google.com");
}
```

Output:

Browser automatically navigates to <https://www.google.com>

Q31. Write a program that sends a customized error message for '404 - Page Not Found'.

Definition / Concept:

web.xml's <error-page> element maps a specific HTTP status code to a custom JSP/HTML page, which is shown instead of the server's default error page.

Program:

```
<!-- web.xml -->
<error-page>
    <error-code>404</error-code>
    <location>/error404.jsp</location>
</error-page>

<!-- error404.jsp -->
<h2>Oops! The page you requested was not found.</h2>
```

Output:

Browser displays:
Oops! The page you requested was not found.

Q32. Write a servlet program that demonstrates the use of URL Rewriting.

Definition / Concept:

URL Rewriting appends the session ID to every link (encodeURL()) so sessions still work even if the browser has cookies disabled.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession();
    String url = res.encodeURL("target");
    res.getWriter().println("<a href='" + url + "'>Click here</a>");
}
```

Output:

Browser shows a link like:

```
<a href="target;jsessionId=3A9F2C1E8B7D">Click here</a>
```

Q33. Write a servlet program that demonstrates the use of Hidden Form Field.

Definition / Concept:

A hidden `<input type='hidden'>` field carries a value along with the form submission without the user seeing or editing it - useful for passing state between pages.

Program:

```
<!-- form.html -->
<form action="hidden" method="post">
    Name: <input name="uname">
    <input type="hidden" name="source" value="RegistrationPage">
    <input type="submit">
</form>

// HiddenServlet.java
res.getWriter().println("Name: " + req.getParameter("uname") +
                        ", Source: " + req.getParameter("source"));
```

Output:

Browser displays:

Name: Ravi, Source: RegistrationPage

Q34. Write a servlet program that checks if a cookie exists; if not, creates one, and displays all cookies sent by the browser.

Definition / Concept:

`req.getCookies()` returns all cookies sent by the browser (or null if none). If the target cookie is missing, a new `Cookie` object is created and attached to the response with `addCookie()`.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    Cookie[] cookies = req.getCookies();
    boolean found = false;
    PrintWriter out = res.getWriter();
    if (cookies != null)
        for (Cookie c : cookies) {
            out.println(c.getName() + " = " + c.getValue());
            if (c.getName().equals("visited")) found = true;
        }
    if (!found) {
        res.addCookie(new Cookie("visited", "yes"));
        out.println("New cookie 'visited' created.");
    }
}
```

Output:

First visit:

New cookie 'visited' created.

Next visit:

visited = yes

Q35. Write a servlet that creates a 'username' cookie (1 day max age), adds it to the response, and displays all cookies received.

Definition / Concept:

setMaxAge(seconds) controls how long a cookie lives in the browser (24*60*60 = 1 day). Cookies are then read back the same way as Q34.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    Cookie c = new Cookie("username", "Ravi");
    c.setMaxAge(24 * 60 * 60);    // 1 day
    res.addCookie(c);

    PrintWriter out = res.getWriter();
    Cookie[] cookies = req.getCookies();
    if (cookies == null) out.println("No cookies were sent.");
    else for (Cookie ck : cookies) out.println(ck.getName() + " = " + ck.getValue());
}
```

Output:

```
username = Ravi
```

Q36. Write a servlet that displays 'Welcome' on first visit and 'Welcome back' on later visits, using Cookie.

Definition / Concept:

The absence of the 'visited' cookie on the first request means it's a new visitor; once created, its presence on later requests identifies a returning visitor.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    Cookie[] cookies = req.getCookies();
    boolean visited = false;
    if (cookies != null)
        for (Cookie c : cookies) if (c.getName().equals("visited")) visited = true;

    PrintWriter out = res.getWriter();
    if (visited) out.println("Welcome back");
    else { res.addCookie(new Cookie("visited", "true")); out.println("Welcome"); }
}
```

Output:

```
First visit : Welcome
Later visits: Welcome back
```

Q37. Develop a servlet that lets the user pick a background color from a dropdown, saves it in a cookie, and applies it to the response page.

Definition / Concept:

The chosen color is read as a parameter, stored in a cookie for future visits, and inserted into an inline CSS style attribute of the response HTML.

Program:

```
<!-- colorform.html -->
<form action="color" method="get">
    <select name="color"><option>red</option><option>green</option><option>blue</option></select>
    <input type="submit">
</form>

// ColorServlet.java
String color = req.getParameter("color");
res.addCookie(new Cookie("bgcolor", color));
res.getWriter().println("<body style='background-color:" + color + "'>Color applied!</body>");
```

Output:

Browser page background changes to the selected color,
with the text: Color applied!

Q38. Write a servlet that displays 'Welcome'/'Welcome back' using HttpSession instead of Cookie.

Definition / Concept:

HttpSession stores data on the SERVER side (unlike a cookie, which is stored in the browser). getSession() creates a new session on first visit; isNew() tells whether this session was just created.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession();
    PrintWriter out = res.getWriter();
    if (session.isNew()) out.println("Welcome");
    else out.println("Welcome back");
}
```

Output:

First visit : Welcome
Later visits: Welcome back

Q39. Write a servlet that accepts a user's name via a form and stores it in an HTTP session; on later visits during the session, greet the user by name.

Definition / Concept:

session.setAttribute() stores the name under a key; session.getAttribute() retrieves it on later requests within the same browser session.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession();
    String name = req.getParameter("name");
    PrintWriter out = res.getWriter();
    if (name != null) session.setAttribute("uname", name);

    Object stored = session.getAttribute("uname");
    out.println(stored != null ? "Hello, " + stored : "Please enter your name.");
}
```

Output:

First request (?name=Ravi): Hello, Ravi
Later requests (same session): Hello, Ravi

Q40. Write a Java Servlet application that authenticates users (login form), stores username in session, shows a personalized welcome page, and implements logout (invalidates session).

Definition / Concept:

On successful login, session.setAttribute() saves the username; the welcome page reads it back with getAttribute(). session.invalidate() destroys the session data for logout.

Program:

```
// LoginServlet.java
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
    String u = req.getParameter("username"), p = req.getParameter("password");
    if (u.equals("admin") && p.equals("admin")) {
        req.getSession().setAttribute("user", u);
    }
}
```

```

        res.sendRedirect("welcome");
    } else res.getWriter().println("Invalid username or password");
}

// WelcomeServlet.java
Object user = req.getSession().getAttribute("user");
res.getWriter().println("Welcome, " + user);

// LogoutServlet.java
req.getSession().invalidate();
res.getWriter().println("You have been logged out.");

```

Output:

```

Welcome, admin
... (after logout) ...
You have been logged out.

```

Q41. Create a servlet that counts how many times a user has visited the page during a session; display the visit count on each load.

Definition / Concept:

A counter stored as a session attribute is read, incremented, and saved back on every request - it persists only for the current browser session.

Program:

```

protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession();
    Integer count = (Integer) session.getAttribute("count");
    count = (count == null) ? 1 : count + 1;
    session.setAttribute("count", count);
    res.getWriter().println("Visit count: " + count);
}

```

Output:

```

Visit count: 1
Visit count: 2
Visit count: 3

```

Q42. Develop a servlet that lets the user pick a background color from a dropdown, saves it in the session, and applies it to the response page.

Definition / Concept:

Same idea as Q37 but using session.setAttribute() instead of a Cookie - the color choice lasts only for the current session, not across browser restarts.

Program:

```

String color = req.getParameter("color");
HttpSession session = req.getSession();
if (color != null) session.setAttribute("bgcolor", color);

Object bg = session.getAttribute("bgcolor");
res.getWriter().println("<body style='background-color:" + bg + "'>Color applied!</body>");

```

Output:

Browser page background is set to the color chosen earlier in the session.

Q43. Write a servlet program that checks if a user has logged in; if not, redirect to a login form. Once logged in, store username in session and display a welcome message.

Definition / Concept:

Checking `session.getAttribute('user') == null` decides whether to force a redirect to the login page or show the protected welcome content - a common access-control pattern.

Program:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession();
    if (session.getAttribute("user") == null) {
        res.sendRedirect("login.jsp");
        return;
    }
    res.getWriter().println("Welcome, " + session.getAttribute("user"));
}
```

Output:

```
Not logged in : browser redirects to login.jsp
Logged in      : Welcome, admin
```

Q44. Write a JSP to print Hello World.

Definition / Concept:

A JSP page mixes HTML with Java. The `<%= expr %>` tag evaluates a Java expression and inserts its value directly into the HTML output.

Program:

```
<html>
<body>
<h1><%= "Hello World" %></h1>
</body>
</html>
```

Output:

```
Browser displays:
Hello World
```

Q45. Write a JSP that accepts Student's details from the HTML page and displays all the information.

Definition / Concept:

The JSP implicit object 'request' (automatically available in every JSP) is used with `getParameter()` to read values submitted by the HTML form.

Program:

```
<!-- studentform.html -->
<form action="showstudent.jsp" method="post">
    Name: <input name="name"> Course: <input name="course">
    <input type="submit">
</form>

<!-- showstudent.jsp -->
<html><body>
Name: <%= request.getParameter("name") %><br>
Course: <%= request.getParameter("course") %>
</body></html>
```

Output:

```
Browser displays:
Name: Raj
Course: BCA
```

Q46. Write a JSP that accepts a year from the user and prints whether it is a leap year or not.

Definition / Concept:

A year is a leap year if divisible by 4 and (not divisible by 100 or divisible by 400) - the classic leap-year rule, written as a scriptlet.

Program:

```
<%
int year = Integer.parseInt(request.getParameter("year"));
boolean leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
%>
<%= year %> is <%= leap ? "a Leap Year" : "not a Leap Year" %>
```

Output:

2024 is a Leap Year

Q47. Write a JSP program to calculate simple interest by accepting values from the user.

Definition / Concept:

Simple Interest = (Principal * Rate * Time) / 100. The three values are parsed from request parameters and used in the formula.

Program:

```
<%
double p = Double.parseDouble(request.getParameter("p"));
double r = Double.parseDouble(request.getParameter("r"));
double t = Double.parseDouble(request.getParameter("t"));
double si = (p * r * t) / 100;
%>
Simple Interest = <%= si %>
```

Output:

Simple Interest = 3000.0

Q48. Write a JSP page that performs division; if divisor is zero, handle the exception using a custom error page.

Definition / Concept:

errorPage='...' declares which JSP handles any uncaught exception. That target page must set isErrorPage='true' to gain access to the implicit 'exception' object.

Program:

```
<!-- divide.jsp -->
<%@ page errorPage="error.jsp" %>
<%
int a = Integer.parseInt(request.getParameter("a"));
int b = Integer.parseInt(request.getParameter("b"));
out.println("Result: " + (a / b));
%>

<!-- error.jsp -->
<%@ page isErrorPage="true" %>
<h3>Error occurred: <%= exception.getMessage() %></h3>
```

Output:

Normal input : Result: 5
Divisor = 0 : Error occurred: / by zero

Q49. Configure an error page in web.xml that catches HTTP 404 errors and displays a user-friendly message.

Definition / Concept:

Same mechanism as Q31 but phrased as JSP: the error-page mapping in web.xml intercepts the 404 status before it reaches the user, showing a friendly JSP instead of the server's raw error trace.

Program:

```
<!-- web.xml -->
<error-page>
    <error-code>404</error-code>
    <location>/notfound.jsp</location>
</error-page>

<!-- notfound.jsp -->
<h3>Sorry, the page you are looking for does not exist.</h3>
```

Output:

Browser displays:
Sorry, the page you are looking for does not exist.

Q50. Write a JSP application that uses <jsp:include> to include reusable pages such as header and footer into a main content page.

Definition / Concept:

<jsp:include> inserts the output of another JSP/HTML file at that point on the page, letting a header/footer be reused across many pages instead of duplicating the markup.

Program:

```
<!-- header.jsp --> <h2>My Website Header</h2>
<!-- footer.jsp --> <p>&copy; 2026 My Website</p>

<!-- main.jsp -->
<jsp:include page="header.jsp" />
<h3>Welcome to the main content area.</h3>
<jsp:include page="footer.jsp" />
```

Output:

Browser displays:
My Website Header
Welcome to the main content area.
(c) 2026 My Website

Q51. Create a JSP application where the user enters a favorite color; use <jsp:forward> in process.jsp to forward to blue.jsp if input is 'blue', otherwise to default.jsp.

Definition / Concept:

<jsp:forward> transfers control entirely to another page on the server (the browser's URL doesn't change). It must be called before any output has been sent to the client.

Program:

```
<!-- process.jsp -->
<%
String color = request.getParameter("color");
if ("blue".equalsIgnoreCase(color)) {
%>
    <jsp:forward page="blue.jsp" />
<% } else { %>
```

```

        <jsp:forward page="default.jsp" />
    <% } %>

    <!-- blue.jsp -->    <h2>You chose Blue!</h2>
    <!-- default.jsp --> <h2>Default page - color not recognized.</h2>

```

Output:

```

Input "blue" : You chose Blue!
Other input  : Default page - color not recognized.

```

Q52. Create a JSP page that uses <jsp:useBean> to instantiate a User JavaBean (name, email, age) and display the user details.

Definition / Concept:

A JavaBean is a plain Java class with private fields and public getter/setter methods. <jsp:useBean> creates (or reuses) an instance, <jsp:setProperty> sets its fields from request parameters, and EL/expression tags display them.

Program:

```

// User.java (JavaBean)
public class User {
    private String name, email; private int age;
    public String getName(){return name;} public void setName(String n){name=n;}
    public String getEmail(){return email;} public void setEmail(String e){email=e;}
    public int getAge(){return age;} public void setAge(int a){age=a;}
}

<!-- user.jsp -->
<jsp:useBean id="user" class="User" />
<jsp:setProperty name="user" property="*" />
Name: <%= user.getName() %>, Email: <%= user.getEmail() %>, Age: <%= user.getAge() %>

```

Output:

```

Name: Ravi, Email: ravi@mail.com, Age: 21

```

Q53. Create a JavaBean EmployeeBean (name, designation, salary); instantiate with <jsp:useBean> scope session, set properties with <jsp:setProperty>, and display using EL (\${}).

Definition / Concept:

scope='session' means the bean instance lives as long as the user's session, so its values persist across multiple JSP pages in that session. Expression Language \${beanName.property} reads a bean's getter without any Java scriptlet code.

Program:

```

// EmployeeBean.java
public class EmployeeBean {
    private String name, designation; private double salary;
    public String getName(){return name;} public void setName(String n){name=n;}
    public String getDesignation(){return designation;} public void setDesignation(String d){designation=d;}
    public double getSalary(){return salary;} public void setSalary(double s){salary=s;}
}

<!-- employee.jsp -->
<jsp:useBean id="employee" class="EmployeeBean" scope="session" />
<jsp:setProperty name="employee" property="name" value="Ravi" />
<jsp:setProperty name="employee" property="designation" value="Manager" />
<jsp:setProperty name="employee" property="salary" value="55000" />

Name: ${employee.name}, Designation: ${employee.designation}, Salary: ${employee.salary}

```

Output:

Name: Ravi, Designation: Manager, Salary: 55000.0

Q54. Write a JSP program to demonstrate the use of application implicit objects - maintain a global visitor counter that increments on every access.

Definition / Concept:

The 'application' implicit object (ServletContext) is shared by ALL users and ALL sessions of the app, unlike 'session' which is per-user - making it perfect for a site-wide visit counter.

Program:

```
<%
Integer count = (Integer) application.getAttribute("visitors");
count = (count == null) ? 1 : count + 1;
application.setAttribute("visitors", count);
%>
Total visitors since server started: <%= count %>
```

Output:

Total visitors since server started: 1
Total visitors since server started: 2 (increments for EVERY user, not per-session)

Q55. Write a JSP program with login (login.jsp), session-based welcome (welcome.jsp), and logout (logout.jsp) that invalidates the session.

Definition / Concept:

This ties together session storage, retrieval, and invalidation across three separate JSP pages - a full login/logout cycle using HttpSession.

Program:

```
<!-- login.jsp -->
<form action="welcome.jsp" method="post">
    Username: <input name="username"> <input type="submit" value="Login">
</form>

<!-- welcome.jsp -->
<% session.setAttribute("user", request.getParameter("username")); %>
Welcome, ${sessionScope.user}! <a href="logout.jsp">Logout</a>

<!-- logout.jsp -->
<% session.invalidate(); %>
You have been logged out. <a href="login.jsp">Login again</a>
```

Output:

Welcome, Ravi! Logout
...
You have been logged out. Login again

Q56. Write a JSP program that stores the user's preferred theme (light/dark) using cookies; retrieve it on subsequent visits and apply it.

Definition / Concept:

A theme name submitted by a form is saved as a Cookie; on later visits the cookie value is read and used in an inline style to redraw the page in that theme.

Program:

```
<%
String theme = request.getParameter("theme");
```

```

if (theme != null) {
    Cookie c = new Cookie("theme", theme);
    response.addCookie(c);
}
String current = "light";
Cookie[] cookies = request.getCookies();
if (cookies != null)
    for (Cookie c : cookies) if (c.getName().equals("theme")) current = c.getValue();
%>
<body style="background:<%= current.equals("dark") ? "#222;color:#fff" : "#fff;color:#000" %>">
Current theme: <%= current %>
</body>

```

Output:

Current theme: dark (page background switches to dark styling)

Q57. Write a JSP that displays 'Welcome' on first visit and 'Welcome back' on later visits, using cookies.

Definition / Concept:

Identical logic to the servlet version (Q36), just written using JSP scriptlets instead of a Java class.

Program:

```

<%
Cookie[] cookies = request.getCookies();
boolean visited = false;
if (cookies != null)
    for (Cookie c : cookies) if (c.getName().equals("visited")) visited = true;
if (!visited) response.addCookie(new Cookie("visited", "true"));
%>
<%= visited ? "Welcome back" : "Welcome" %>

```

Output:

First visit : Welcome
 Later visits: Welcome back

Q58. Write a JSP program to display current date and time using JSP Expression Language (EL).

Definition / Concept:

EL cannot call constructors directly, so a scriptlet first stores a Date object as a request attribute; EL (\${today}) then simply displays that attribute's toString() value.

Program:

```

<% request.setAttribute("today", new java.util.Date()); %>
Current Date/Time: ${today}

```

Output:

Current Date/Time: Sun Sep 13 16:52:10 IST 2026

Q59. Write a JSP application using EL for dynamic content across pages: index.jsp takes a name, passes it via request to welcome.jsp, which displays it using EL.

Definition / Concept:

\${param.name} is EL's shortcut for request.getParameter('name') - it reads a submitted form field directly without any scriptlet.

Program:

```

<!-- index.jsp -->
<form action="welcome.jsp" method="get">
    Name: <input name="name"> <input type="submit">
</form>

<!-- welcome.jsp -->
Hello, ${param.name}! Welcome to the site.

```

Output:

Hello, Ravi! Welcome to the site.

Q60. Write a JSP program that calculates addition, subtraction, multiplication and division using Expression Language and displays results dynamically.

Definition / Concept:

EL directly supports arithmetic operators (+, -, *, / or div, % or mod) on parameter values, so no Java scriptlet is required at all.

Program:

```

Sum: ${param.a + param.b}<br>
Difference: ${param.a - param.b}<br>
Product: ${param.a * param.b}<br>
Quotient: ${param.a / param.b}

```

Output:

```

Sum: 13
Difference: 7
Product: 30
Quotient: 3.3333333333333335

```

Q61. Write a JSP program to check whether it is a leap year or not using JSTL.

Definition / Concept:

The JSTL core tag library (prefix c) replaces scriptlets with XML-like tags. <c:set> stores a computed boolean, and <c:if>/<c:choose> branch based on it - keeping JSP pages free of embedded Java.

Program:

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="year" value="${param.year}" />
<c:set var="leap" value="${(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)}" />
<c:choose>
    <c:when test="${leap}">${year} is a Leap Year</c:when>
    <c:otherwise>${year} is not a Leap Year</c:otherwise>
</c:choose>

```

Output:

2024 is a Leap Year

Q62. Write a JSP program to check whether it is an even or odd number using JSTL.

Definition / Concept:

<c:if test='...'> only renders its body when the EL condition is true - the % (mod) operator checks divisibility by 2.

Program:

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="n" value="${param.num}" />
<c:if test="${n % 2 == 0}">${n} is Even</c:if>
<c:if test="${n % 2 != 0}">${n} is Odd</c:if>

```

Output:

7 is Odd

Q63. Write a JSP program to print prime numbers from 1 to 100 using JSTL.

Definition / Concept:

<c:forEach> loops like a for-loop in EL/JSTL. For each candidate number, an inner loop checks divisibility; <c:set> tracks whether any divisor was found, similar to a boolean flag in plain Java.

Program:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:forEach var="n" begin="2" end="100">
  <c:set var="isPrime" value="true" />
  <c:forEach var="i" begin="2" end="{n - 1}">
    <c:if test="{n % i == 0}"><c:set var="isPrime" value="false" /></c:if>
  </c:forEach>
  <c:if test="{isPrime}">{n} </c:if>
</c:forEach>
```

Output:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Q64. Write a JSP program to display whether a number is divisible by 2 using JSTL.

Definition / Concept:

A simple JSTL if-check on the modulo result - functionally the same test as Q62, phrased as a divisibility check rather than even/odd wording.

Program:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="n" value="{param.num}" />
<c:choose>
  <c:when test="{n % 2 == 0}">{n} is divisible by 2</c:when>
  <c:otherwise>{n} is NOT divisible by 2</c:otherwise>
</c:choose>
```

Output:

10 is divisible by 2

Q65. Write a program with a Servlet that accepts Username as input; use a JSP page to welcome the user. Use the MVC pattern to separate concerns.

Definition / Concept:

MVC (Model-View-Controller) splits responsibilities: the Servlet (Controller) receives the request and decides what to do, a simple Model class/attribute holds the data, and the JSP (View) is responsible only for displaying it - never for processing logic.

Program:

```
<!-- index.html (View - input form) -->
<form action="welcome" method="post">
  Username: <input name="username"> <input type="submit">
</form>

// WelcomeServlet.java (Controller)
protected void doPost(HttpServletRequest req, HttpServletResponse res)
  throws ServletException, IOException {
  req.setAttribute("username", req.getParameter("username")); // Model data
  req.getRequestDispatcher("welcome.jsp").forward(req, res); // hand off to View
```

```

}

<!-- welcome.jsp (View) -->
Welcome, ${username}!

```

Output:

Welcome, Ravi!

Q66. Write a login program (admin/admin) using MVC with 2 separate views - one for success, one for failure.

Definition / Concept:

The Controller Servlet checks credentials and forwards to whichever View JSP matches the outcome, keeping the decision logic completely out of the JSP files.

Program:

```

// LoginServlet.java (Controller)
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException {
    String u = req.getParameter("username"), p = req.getParameter("password");
    if (u.equals("admin") && p.equals("admin"))
        req.getRequestDispatcher("success.jsp").forward(req, res);
    else
        req.getRequestDispatcher("failure.jsp").forward(req, res);
}

<!-- success.jsp --> <h2>Welcome, Admin!</h2>
<!-- failure.jsp --> <h2>Incorrect username or password.</h2>

```

Output:

Correct : Welcome, Admin!
 Incorrect : Incorrect username or password.

Q67. Write a login program (Table: Login, fields username/password) using MVC, fetching credentials from the database.

Definition / Concept:

The Model layer here is the database query itself (via JDBC); the Controller Servlet uses its result to decide which View to forward to, keeping SQL code out of the JSP.

Program:

```

// LoginServlet.java (Controller + Model access)
protected void doPost(HttpServletRequest req, HttpServletResponse res)
    throws Exception {
    String u = req.getParameter("username"), p = req.getParameter("password");
    Connection con = DBUtil.getConnection();
    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM Login WHERE username=? AND password=?");
    ps.setString(1, u); ps.setString(2, p);
    ResultSet rs = ps.executeQuery();
    if (rs.next())
        req.getRequestDispatcher("success.jsp").forward(req, res);
    else
        req.getRequestDispatcher("failure.jsp").forward(req, res);
}

```

Output:

Valid DB credentials : forwards to success.jsp -> Welcome!
 Invalid DB credentials : forwards to failure.jsp -> Incorrect username or password.

Q68. Develop a Registration module using JSP, Servlet and MVC with database connectivity (Table: User - username, password, mobile).

Definition / Concept:

The View (registration form) collects data, the Controller (Servlet) validates and inserts it via JDBC (Model), then forwards to a confirmation View - a complete MVC registration flow.

Program:

```
<!-- register.jsp (View) -->
<form action="register" method="post">
    Username: <input name="username"> Password: <input name="password">
    Mobile: <input name="mobile"> <input type="submit">
</form>

// RegisterServlet.java (Controller)
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws Exception {
    Connection con = DBUtil.getConnection();
    PreparedStatement ps = con.prepareStatement("INSERT INTO User VALUES(?,?,?)");
    ps.setString(1, req.getParameter("username"));
    ps.setString(2, req.getParameter("password"));
    ps.setString(3, req.getParameter("mobile"));
    ps.executeUpdate();
    req.getRequestDispatcher("success.jsp").forward(req, res);
}
```

Output:

Browser (success.jsp) displays: Registration successful!

Q69. Write a program that lets a user request a list of students by course and semester; use a Servlet to retrieve data from a Model and display it dynamically on a JSP page. (Table: Student)

Definition / Concept:

The Servlet (Controller) queries the database (Model) and stores the resulting list as a request attribute; the JSP (View) loops over that list with JSTL's <c:forEach> to render an HTML table.

Program:

```
// StudentServlet.java (Controller + Model access)
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws Exception {
    Connection con = DBUtil.getConnection();
    PreparedStatement ps = con.prepareStatement(
        "SELECT * FROM Student WHERE course=? AND semester=?");
    ps.setString(1, req.getParameter("course"));
    ps.setInt(2, Integer.parseInt(req.getParameter("semester")));
    req.setAttribute("students", ps.executeQuery());
    req.getRequestDispatcher("list.jsp").forward(req, res);
}

<!-- list.jsp (View) -->
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
<c:forEach var="s" items="${students.rows}">
    ${s.rollno} - ${s.firstname} ${s.lastname}<br>
</c:forEach>
```

Output:

```
1 - Raj Patel
2 - Meera Shah
```

Q70. Write a Contact Management System using MVC where users can add contacts (name, phone) and view all saved contacts.

Definition / Concept:

The Model (Contact class + an in-memory or DB-backed list) holds the data; the Controller Servlet processes 'add' submissions and updates the Model; the View JSP displays the current contact list.

Program:

```
// Contact.java (Model)
class Contact { String name, phone; Contact(String n,String p){name=n;phone=p;} }

// ContactServlet.java (Controller)
static java.util.List<Contact> contacts = new java.util.ArrayList<>();
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws Exception {
    contacts.add(new Contact(req.getParameter("name"), req.getParameter("phone")));
    req.setAttribute("contacts", contacts);
    req.getRequestDispatcher("list.jsp").forward(req, res);
}

<!-- list.jsp (View) -->
<c:forEach var="c" items="${contacts}">${c.name} - ${c.phone}<br></c:forEach>
```

Output:

```
Ravi - 9876543210
Meera - 9998887770
```

Q71. Write an MVC application using Servlet, JSP and JavaBean to insert and display employee data (emp table: empno, empnm, designation, dept).

Definition / Concept:

The JavaBean (Model) is a plain data holder; the Servlet (Controller) populates the bean from form input and saves it via JDBC; the JSP (View) displays it using <jsp:useBean> and EL.

Program:

```
// EmployeeBean.java (Model)
public class EmployeeBean {
    private int empno; private String empnm, designation, dept;
    // ... standard getters and setters for each field ...
}

// EmpServlet.java (Controller)
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws Exception {
    PreparedStatement ps = DBUtil.getConnection().prepareStatement("INSERT INTO emp VALUES(?,?,?,?)");
    ps.setInt(1, Integer.parseInt(req.getParameter("empno")));
    ps.setString(2, req.getParameter("empnm"));
    ps.setString(3, req.getParameter("designation"));
    ps.setString(4, req.getParameter("dept"));
    ps.executeUpdate();
    req.getRequestDispatcher("display.jsp").forward(req, res);
}

<!-- display.jsp (View) -->
Employee added: ${param.empnm} - ${param.designation}
```

Output:

```
Employee added: Ravi - Manager
```

Q72. Develop a full CRUD application using MVC (Servlet, JSP, JavaBean) for student records (studnt table: rollno, name, course, semester).

Definition / Concept:

One Controller Servlet routes to different logic (insert/update/delete/list) based on an 'action' parameter, each using the Bean/JDBC Model, and forwards to the matching View JSP (add/edit/list).

Program:

```
// StudentServlet.java (Controller) - action-based routing
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws Exception {
    String action = req.getParameter("action");
    Connection con = DBUtil.getConnection();
    switch (action) {
        case "insert": {
            PreparedStatement ps = con.prepareStatement("INSERT INTO studnt VALUES(?,?,?,?)");
            ps.setInt(1, Integer.parseInt(req.getParameter("rollno")));
            ps.setString(2, req.getParameter("name"));
            ps.setString(3, req.getParameter("course"));
            ps.setInt(4, Integer.parseInt(req.getParameter("semester")));
            ps.executeUpdate(); break;
        }
        case "update": { /* similar PreparedStatement UPDATE ... WHERE rollno=? */ break; }
        case "delete": { /* similar PreparedStatement DELETE ... WHERE rollno=? */ break; }
    }
    req.getRequestDispatcher("list.jsp").forward(req, res); // View shows updated list
}
```

Output:

list.jsp (View) displays the current student table after every insert / update / delete action.

Q73. Create a simple Hibernate application connecting to MySQL: define a Student entity (rollno, name, course, semester), save a new object and retrieve it using a session.

Definition / Concept:

Hibernate is an ORM (Object-Relational Mapping) tool - it maps a Java class (@Entity) directly to a database table, so objects can be saved/loaded without writing SQL. hibernate.cfg.xml configures the DB connection.

Program:

```
// Student.java (Entity)
@Entity
public class Student {
    @Id private int rollno;
    private String name, course; private int semester;
    // getters and setters
}

// Main.java
SessionFactory sf = new Configuration().configure().buildSessionFactory();
Session session = sf.openSession();
session.beginTransaction();

Student s = new Student();
s.setRollno(1); s.setName("Raj"); s.setCourse("BCA"); s.setSemester(5);
session.save(s);
session.getTransaction().commit();

Student loaded = session.get(Student.class, 1);
System.out.println(loaded.getName() + " - " + loaded.getCourse());
```

Output:

Raj - BCA

Q74. Create an Employee entity (empno, empnm, department); use Hibernate to insert a record, then retrieve and display all employees.

Definition / Concept:

session.createQuery('from Employee') runs HQL (Hibernate Query Language), which queries in terms of Java class/field names instead of raw table/column names.

Program:

```
@Entity
public class Employee {
    @Id private int empno; private String empnm, department;
    // getters and setters
}

Session session = sf.openSession();
session.beginTransaction();
Employee e = new Employee(); e.setEmpno(1); e.setEmpnm("Ravi"); e.setDepartment("IT");
session.save(e);
session.getTransaction().commit();

List<Employee> list = session.createQuery("from Employee", Employee.class).list();
for (Employee emp : list) System.out.println(emp.getEmpnm() + " - " + emp.getDepartment());
```

Output:

Ravi - IT

Q75. Create a Hibernate application with a Product entity (id, name, price); use HQL to fetch all products and display id, name, price.

Definition / Concept:

HQL statements ('from Product') look like SQL but operate on entity classes/properties, and Hibernate translates them into the correct SQL for the underlying database automatically.

Program:

```
@Entity
public class Product {
    @Id private int id; private String name; private double price;
    // getters and setters
}

List<Product> products = session.createQuery("from Product", Product.class).list();
for (Product p : products)
    System.out.println(p.getId() + " " + p.getName() + " " + p.getPrice());
```

Output:

1 Pen 10.0
2 Notebook 40.0

Q76. Create a simple login application using the Spring framework with a LoginController managing authentication via Spring MVC, with login form, success and failure templates.

Definition / Concept:

@Controller marks the class as a Spring MVC handler; @PostMapping maps the form submission to a method, which returns the logical view NAME (a template) rather than writing HTML directly - Spring resolves the actual template file.

Program:

```
@Controller
public class LoginController {

    @GetMapping("/login")
    public String loginForm() { return "login"; }           // login.html template

    @PostMapping("/login")
```

```

        public String doLogin(@RequestParam String username,
                               @RequestParam String password) {
            if (username.equals("admin") && password.equals("admin"))
                return "success";        // success.html template
            return "failure";            // failure.html template
        }
    }
}

```

Output:

Correct credentials : renders success.html - "Login successful!"
 Incorrect credentials: renders failure.html - "Invalid username or password"

Q77. Develop an application using Spring Boot and Spring Data JPA to insert and display employee data (Employee table: empno, empnm, designation, dept).

Definition / Concept:

@Entity + a Repository interface (extending JpaRepository) gives free save()/findAll() methods with NO SQL or boilerplate DAO code at all - Spring Data JPA generates the implementation automatically.

Program:

```

@Entity
public class Employee {
    @Id private int empno; private String empnm, designation, dept;
    // getters and setters
}

public interface EmployeeRepository extends JpaRepository<Employee, Integer> { }

@RestController
public class EmployeeController {
    @Autowired EmployeeRepository repo;

    @PostMapping("/employees")
    public Employee add(@RequestBody Employee e) { return repo.save(e); }

    @GetMapping("/employees")
    public List<Employee> all() { return repo.findAll(); }
}

```

Output:

POST /employees -> {"empno":1,"empnm":"Ravi","designation":"Manager","dept":"IT"}
 GET /employees -> [{"empno":1,"empnm":"Ravi","designation":"Manager","dept":"IT"}]

Q78. Develop an application using Spring Boot and Spring Data JPA to perform CRUD operations on a Student table (rollno, name, course, semester).

Definition / Concept:

JpaRepository already provides save() (create/update), findAll()/findById() (read) and deleteById() (delete) - so full CRUD needs only a thin REST controller on top, no manual SQL anywhere.

Program:

```

@Entity
public class Student {
    @Id private int rollno; private String name, course; private int semester;
    // getters and setters
}

public interface StudentRepository extends JpaRepository<Student, Integer> { }

@RestController
@RequestMapping("/students")

```

```

public class StudentController {
    @Autowired StudentRepository repo;

    @PostMapping public Student create(@RequestBody Student s) { return repo.save(s); }
    @GetMapping public List<Student> readAll() { return repo.findAll(); }
    @PutMapping("/{id}") public Student update(@PathVariable int id, @RequestBody Student s) {
        s.setRollno(id); return repo.save(s);
    }
    @DeleteMapping("/{id}") public void delete(@PathVariable int id) { repo.deleteById(id); }
}

```

Output:

```

POST    /students    -> creates a new student record
GET      /students    -> returns the full student list as JSON
PUT      /students/1  -> updates student with rollno 1
DELETE  /students/1  -> removes student with rollno 1

```